

Proyecto CineMax V2

Memoria Técnica y Documentación del Sistema



Alumno: Rafael Hidalgo Torres

Profesor: Amin Harou Azouagh

Asignatura: Acceso a Datos

Curso: Desarrollo de Aplicaciones Multiplataforma (DAM)

Índice

1. Introducción y Objetivos
2. Arquitectura del Sistema
 - 2.1. Backend (Spring Boot)
 - 2.2. Frontend (React)
 - 2.3. Base de Datos (PostgreSQL)
3. Seguridad y Autenticación (RBAC + JWT)
 - 3.1. Niveles de Acceso y Roles
 - 3.2. Configuración de Filtros y Permisos
4. Desarrollo del Backend
 - 4.1. Patrón de Diseño (Capas)
 - 4.2. DTOs y MapStruct
5. Desarrollo del Frontend
 - 5.1. Interfaz de Usuario (UI/UX)
 - 5.2. Integración con TMDB API
 - 5.3. Sistema de Fallback de Imágenes
6. Despliegue y Pruebas
7. Conclusiones

1. Introducción y Objetivos

El presente documento detalla la arquitectura, diseño y desarrollo del **Proyecto CineMax V2**. Este proyecto es una aplicación web integral (Full-Stack) diseñada para la gestión de un cine moderno, permitiendo la administración de la cartelera, la compra de entradas y la gestión de usuarios.

El objetivo principal de la práctica para la asignatura de *Acceso a Datos* ha sido construir una API REST robusta que se comunique de manera eficiente con una base de datos relacional y que garantice la seguridad de las transacciones mediante un sistema moderno de autenticación y autorización.

2. Arquitectura del Sistema

El proyecto se divide en tres bloques fundamentales que se comunican entre sí de forma desacoplada:

2.1. Backend (Spring Boot)

Desarrollado en Java 21, utilizando el framework Spring Boot. Actúa como el núcleo lógico del sistema. Provee una API RESTful para el consumo de datos y operaciones. Se encarga de la lógica de negocio, la validación de datos, la persistencia y la seguridad del sistema.

2.2. Frontend (React)

Construido como una Single Page Application (SPA) utilizando React. Interacciona con el backend mediante peticiones HTTP asíncronas (Fetch/Axios). Destaca por un diseño responsivo y moderno, enfocado en ofrecer una experiencia de usuario (UX) fluida, sin recargas de página.

2.3. Base de Datos (PostgreSQL)

Se ha elegido PostgreSQL como Sistema Gestor de Bases de Datos Relacionales (SGBDR) debido a su rendimiento y robustez. El modelo relacional incluye tablas para Usuarios, Roles, Películas, Salas, Funciones y Ventas, con sus respectivas restricciones de integridad referencial.

3. Seguridad y Autenticación (RBAC + JWT)

Uno de los hitos técnicos más importantes del proyecto es la implementación de seguridad avanzada utilizando **JSON Web Tokens (JWT)** y control de acceso basado en roles (**RBAC**).

3.1. Niveles de Acceso y Roles

El sistema divide el acceso en tres niveles diferenciados:

Nivel	Descripción	Permisos Típicos
Público	Usuarios no autenticados	Ver la cartelera de películas, horarios y detalles públicos.
USUARIO	Clientes registrados	Comprar entradas y ver su propio historial de ventas (Ownership).
ADMINISTRADOR	Gestores del cine	CRUD completo de Salas, Películas, Funciones. Gestión de todo el sistema.

3.2. Configuración de Filtros y Permisos

La seguridad en Spring Boot se ha configurado de dos maneras complementarias:

1. **SecurityConfig:** Define filtros globales a nivel de rutas. Se permite el acceso público a endpoints como `/api/v1/auth/**`, `/api/v1/peliculas (GET)`, etc., denegando todo lo demás si no se porta un JWT válido.
2. **@PreAuthorize:** Se ha utilizado seguridad a nivel de método. Por ejemplo, para la creación de películas:

```
@PostMapping
@PreAuthorize("hasRole('ADMINISTRADOR')")
public ResponseEntity<PelículaOutputDTO> create(...) {
    ...
}
```

3. **Lógica de Pertenencia (Ownership):** A nivel de servicio, se valida que un `USUARIO` solo pueda acceder a recursos que le pertenecen (por ejemplo, el endpoint `/mis-ventas` o el detalle de una venta propia), evitando accesos no autorizados a datos de otros clientes.

4. Desarrollo del Backend

4.1. Patrón de Diseño (Capas)

El código de Spring Boot sigue una arquitectura limpia basada en capas:

- **Controladores (Controllers):** Gestionan las peticiones HTTP, extraen los datos y devuelven respuestas adecuadas (Status Codes y JSON).
- **Servicios (Services):** Contienen toda la lógica de negocio y las reglas de seguridad específicas.
- **Repositorios (Repositories):** Interfaces que extienden de `JpaRepository`, gestionando las transacciones contra PostgreSQL usando Spring Data JPA.
- **Entidades (Entities):** Clases mapeadas a la base de datos mediante anotaciones ORM (Hibernate).

4.2. DTOs y MapStruct

Para evitar exponer directamente las entidades de base de datos e incurrir en problemas de recursión o fuga de datos sensibles, se ha implementado el patrón **Data Transfer Object (DTO)**.

Se utiliza la librería **MapStruct** para generar automáticamente las implementaciones de conversión (mapeo) entre Entidades y DTOs, y la librería **Lombok** para reducir el código repetitivo (getters, setters, constructores).

5. Desarrollo del Frontend

5.1. Interfaz de Usuario (UI/UX)

Se ha diseñado una interfaz con estética "Ultra Premium" y cinematográfica. Se han empleado técnicas modernas de CSS como *Glassmorphism* (desenfoques dinámicos usando `backdrop-filter`), fondos dinámicos y un layout estructurado y responsivo mediante **CSS Grid**.

5.2. Integración con TMDB API

Para garantizar que el catálogo luzca profesional, las portadas (posters) y fondos de las películas (backdrops) se consumen directamente de **The Movie Database (TMDB)** en alta resolución (hasta 1280p). Esto mejora drásticamente el aspecto visual de la aplicación.

5.3. Sistema de Fallback de Imágenes

Se ha desarrollado un mecanismo inteligente en el código frontend: si una URL de imagen falla o se encuentra caída, el sistema inyecta automáticamente un componente visual (un SVG dorado o *placeholder* generado por código) para que el diseño en cuadrícula nunca se rompa.

6. Despliegue y Pruebas

El entorno está preparado para ser fácilmente evaluable:

- El esquema de base de datos se genera automáticamente por Hibernate (`update` o `create` según configuración).
- Se incluye un archivo **Cine_V2_API.postman_collection.json** con todos los endpoints preparados para ser importados en Postman, facilitando las pruebas de la API (incluyendo la inyección de JWT).
- El código está debidamente gestionado y alojado en la rama principal del repositorio de control de versiones.

7. Conclusiones

El proyecto CineMax V2 cumple y excede los requerimientos planteados para la asignatura de Acceso a Datos. Se ha logrado desarrollar una plataforma que no solo resuelve el problema técnico del acceso, mapeo y persistencia de la información, sino que además envuelve todo en una capa de seguridad sólida (RBAC + JWT) y se presenta mediante una interfaz cliente de alta calidad profesional.

El uso conjunto de tecnologías empresariales como Spring Boot, PostgreSQL, React y MapStruct demuestra la asimilación de los conceptos impartidos durante el curso.